

OSSP PRACTICAL 8

```
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ mkdir practical8
alurusaranya@LAPTOP-0PA61QEN:~$ cd practical8
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ nano memory.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ gcc -g memory.c -o me
memory
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ ls
memory  memory.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ ./memory
Enter number of elements: 10
Enter 10 elements:
1
2
3
4
5
6
7
8
9
10
Elements are:
1 2 3 4 5 6 7 8 9 10
Memory released successfully.
alurusaranya@LAPTOP-0PA61QEN:~/practical8$
```

```

GNU nano 7.2                                memory.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;
    int *arr;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    arr = (int *)malloc(n * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Elements are:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}

```

```

alurusaranya@LAPTOP-0PA61QEN:~/practical8$ nano memory.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ valgrind --leak-check
=full --show-leak-kinds=all ./memory
==1670== Memcheck, a memory error detector
==1670== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Sewar
d et al.
==1670== Using Valgrind-3.22.0 and LibVEX; rerun with -h for cop
yright info
==1670== Command: ./memory
==1670==
Enter number of elements: 3
Enter 3 elements:
10
20
30
Elements are:
10 20 30
Memory released successfully.
==1670==
==1670== HEAP SUMMARY:
==1670==      in use at exit: 0 bytes in 0 blocks
==1670==    total heap usage: 3 allocs, 3 frees, 2,060 bytes allo
cated
==1670==
==1670== All heap blocks were freed -- no leaks are possible
==1670==
==1670== For lists of detected and suppressed errors, rerun with
: -s
==1670== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ cp memory.c leak.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ nano leak.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ |

```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;
    int *arr;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    arr = (int *)malloc(n * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter %d elements:\n", n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Elements are:\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

```
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Elements are:\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    free(arr);
    arr = NULL;

    printf("Memory released successfully.\n");

    return 0;
}
```

```

alurusaranya@LAPTOP-0PA61QEN:~/practical8$ cp memory.c leak.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ nano leak.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ nano memory.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ gcc -g memory.c -o memory
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ valgrind --leak-check=full --show-leak-kinds=all ./memory
==1691== Memcheck, a memory error detector
==1691== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1691== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1691== Command: ./memory
==1691==
Enter number of elements: 2
Enter 2 elements:
10 11
Elements are:
10 11
Memory released successfully.
==1691==
==1691== HEAP SUMMARY:
==1691==    in use at exit: 0 bytes in 0 blocks
==1691== total heap usage: 3 allocs, 3 frees, 2,056 bytes allocated
==1691==
==1691== All heap blocks were freed -- no leaks are possible
==1691==
==1691== For lists of detected and suppressed errors, rerun with: -s
==1691== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ ls -l
total 28
-rw-rw-r-- 1 alurusaranya alurusaranya 622 Sep 18 08:03 leak.c
-rwxrwxr-x 1 alurusaranya alurusaranya 17928 Sep 18 08:05 memory
-rw-rw-r-- 1 alurusaranya alurusaranya 644 Sep 18 08:05 memory.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ valgrind --leak-check=full --show-leak-kinds=all ./memory
==1697== Memcheck, a memory error detector
==1697==
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ ls -l
total 28
-rw-rw-r-- 1 alurusaranya alurusaranya 622 Sep 18 08:03 leak.c
-rwxrwxr-x 1 alurusaranya alurusaranya 17928 Sep 18 08:05 memory
-rw-rw-r-- 1 alurusaranya alurusaranya 644 Sep 18 08:05 memory.c
alurusaranya@LAPTOP-0PA61QEN:~/practical8$ valgrind --leak-check=full --show-leak-kinds=all ./memory
==1697== Memcheck, a memory error detector
==1697== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1697== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1697== Command: ./memory
==1697==
Enter number of elements: 1
Enter 1 elements:
10
Elements are:
10
Memory released successfully.
==1697==
==1697== HEAP SUMMARY:
==1697==    in use at exit: 0 bytes in 0 blocks
==1697== total heap usage: 3 allocs, 3 frees, 2,052 bytes allocated
==1697==
==1697== All heap blocks were freed -- no leaks are possible
==1697==
==1697== For lists of detected and suppressed errors, rerun with: -s
==1697== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
alurusaranya@LAPTOP-0PA61QEN:~/practical8$

```

Week 8 – Theory: Dynamic Memory Allocation & Valgrind

1. Dynamic Memory Allocation

Dynamic memory allocation means allocating memory while the program is running, rather than fixing the memory size beforehand.

In C, the main functions are:

- `malloc()` – allocates memory.
- `calloc()` – allocates and initializes memory to zero.
- `realloc()` – changes the size of previously allocated memory.
- `free()` – releases allocated memory.

2520080012

Example:

```
int *p = malloc(sizeof(int));
```

```
*p = 10;
```

```
free(p);
```

2. Heap Memory

Dynamic memory is allocated from the **heap**.

The programmer is responsible for releasing this memory using `free()`.

If allocated memory is not released, it can cause a **memory leak**.

3. Memory Leak

A **memory leak** occurs when a program allocates memory but loses the reference to it without freeing the memory.

Example:

```
int *p = malloc(100 * sizeof(int));
```

```
p = NULL;
```

The allocated memory can no longer be accessed or freed → **memory leak**.

4. Valgrind

Valgrind is a tool used to detect memory-related problems in C/C++ programs.

It can help identify:

- Memory leaks
- Invalid memory access
- Use of uninitialized memory
- Incorrect memory deallocation

The handout specifically includes running Valgrind, identifying leaks, fixing memory errors, releasing resources, and verifying cleanup.

Basic command:

```
valgrind --leak-check=full ./program
```

5. Invalid Memory Access

2520080012

This happens when a program accesses memory that it should not access.

:

```
int *p = malloc(sizeof(int));
```

```
free(p);
```

```
*p = 10;
```

Here p is used **after it has been freed**.

This is called a **use-after-free** error. Example

6. Importance of free()

Whenever dynamically allocated memory is no longer required:

```
free(pointer);
```

should be used.

Good practice:

```
free(p);
```

```
p = NULL;
```

Setting the pointer to NULL helps avoid accidentally using the old address.